

计算机体系结构第三次实验报告-例外中断

2312966 林晖鹏

一、实验要求

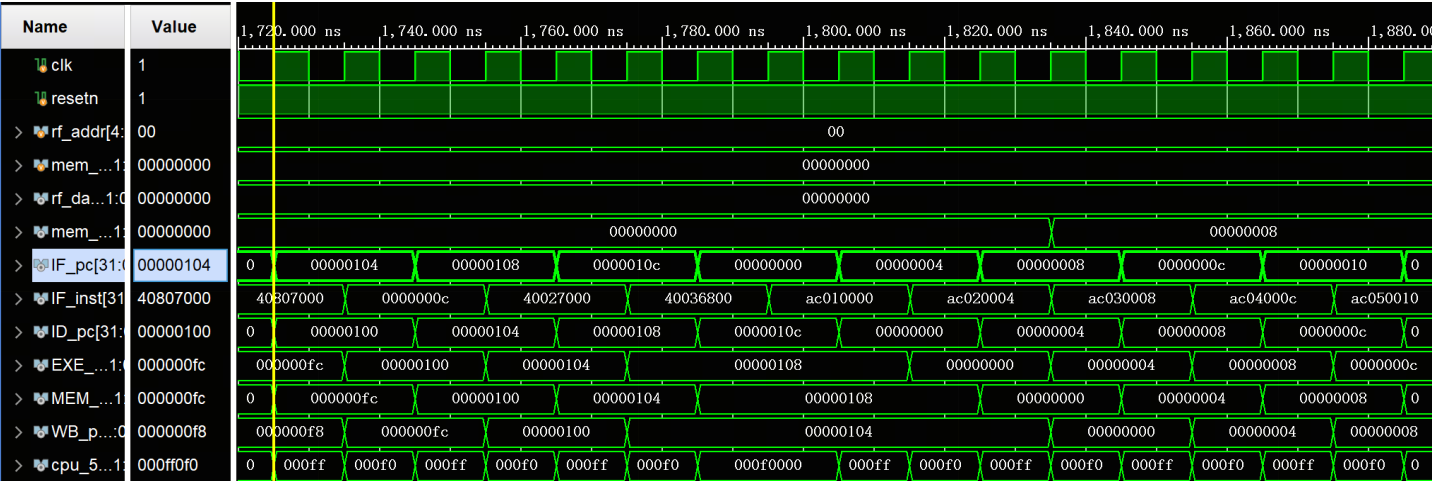
- 1. 自行修改inst_rom中的指令，复现例外
- 2. 实现对应的例外支持
- 3. 实验报告中体现实现支持之前的波形，和之后的波形，并详细分析实验结果。

二、复现现有例外

在之前实验完成的基础上，我们目前在五级流水 CPU 中已经支持了部分例外/返回机制，主要包括 SYSCALL 例外指令、ERET 返回指令以及与之配合的 CP0 寄存器（STATUS、CAUSE、EPC）的基本功能。

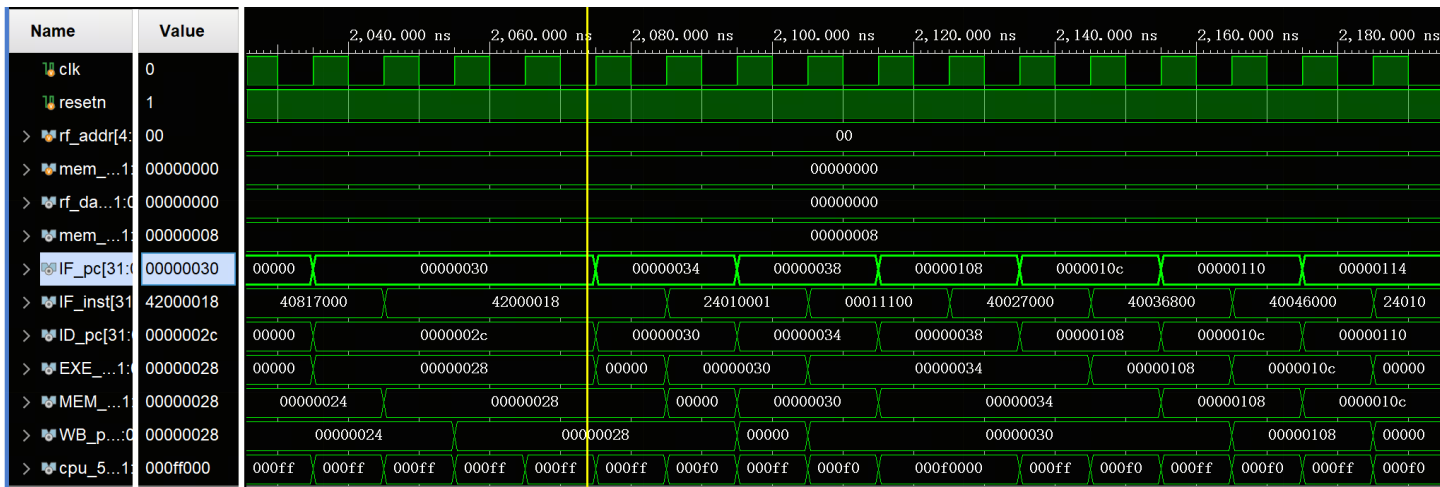
1. syscall例外指令

在原指令集中，当执行到pc=104的时候，会执行syscall指令，然后跳转到pc=0处，交由所谓的内核态，跳转至其他地方来执行指令，如下图所示：



2. ERET 返回指令

而ERET指令是返回syscall指令执行之前的位置继续执行指令，可以从下图看到，从pc=30处取出eret指令之后，后面重新跳转到之前执行的syscall指令的下一个指令位置，也就是pc=104的下一条pc=108。



三、添加额外的例外指令支持

本实验在保持原有 SYSCALL/ERET/CP0 框架的基础上，新增三个典型异常：整数溢出、地址错、BREAK，以及未实现/保留指令（Reserved Instruction）。以下仅描述关键实现点与执行流程。

1. 整数溢出

触发条件：当执行有符号加/减法 ADD、ADDI、SUB 时发生溢出（结果符号与操作数符号不一致）。

- 产生位置与信号：
 - 在 decode.v 识别 `ADD/ADDI/SUB` 并生成 `ov\_en` 标志，随 `ID\_EXE\_bus` 下传。

代码块

```
1 wire ov_en;
2 assign ov_en = inst_ADD | inst_ADDI | inst_SUB;
```

- 在 `exe.v` 根据 `alu\_control` 与两个操作数做有符号溢出检测，形成 `overflow` 标志，并随 `EXE\_MEM\_bus` 传递。

代码块

```
1 wire overflow = ov_en & ( (alu_add_op & (~
  (alu_operand1[31]^alu_operand2[31])) & (alu_operand1[31]^ov_sum[31])) |
  (alu_sub_op & ( (alu_operand1[31]^alu_operand2[31])) &
  (alu_operand1[31]^ov_sum[31])) );
```

- 异常确认 (in WB) :
 - `wb.v` 在 WB 阶段汇总异常来源，若 `overflow\_ex=1`，则：
 - `CAUSE.ExcCode <= 12`（整数溢出）；

- `EPC <= 当前异常指令的 PC`;
- `STATUS.EXL <= 1`;

代码块

```

1  else if (overflow_ex)
2      cause_exc_code_r <= 5'd12;    // Integer Overflow
3      ...
4  reg [31:0] epc_r;
5      assign cp0r_epc = epc_r;
6      always @(posedge clk)
7          begin
8              if (syscall | break_ex | reserved_ex | overflow_ex | addr_err_load |
addr_err_store)
9                  begin
10                     epc_r <= pc;
11                 end
12                 else if (epc_wen)
13                     begin
14                         epc_r <= mem_result;
15                     end
16                 end
17             ...
18         else if (syscall | break_ex | reserved_ex | overflow_ex | addr_err_load |
addr_err_store)
19             begin
20                 status_exl_r <= 1'b1;
21             end

```

- 通过 `exc\_bus` 向 IF 发出异常入口地址 `EXC\_ENTER\_ADDR(0x0)`，并产生 `cancel` 清空流水线。

代码块

```

1  assign exc_bus = {exc_valid,exc_pc};

```

- 返回：异常处理程序中用 `MFC0 EPC -> EPC+4 -> MTC0 EPC -> ERET` 返回到触发指令的下一条继续执行。

2. 地址错误

触发条件：对字访问（LW/SW）时地址未 4 字节对齐（`addr[1:0] != 2'b00`）。

- 产生位置与信号：
 - 在 `mem.v`，已取得访存地址 `dm\_addr` 和控制 `inst\_load/inst\_store/ls\_word`；
 - 组合产生 `addr\_err\_load`（对齐错误的 load）与 `addr\_err\_store`（对齐错误的 store），并随 `MEM\_WB\_bus` 传递。

代码块

```
1  wire addr_err_load  = MEM_valid & inst_load  & ls_word & (dm_addr[1:0] !=
    2'b00);
2  wire addr_err_store = MEM_valid & inst_store & ls_word & (dm_addr[1:0] !=
    2'b00);
```

- 异常确认：
 - `wb.v` 优先级处理 CAUSE：
 - `addr\_err\_load` -> `ExcCode=4`；`addr\_err\_store` -> `ExcCode=5`；

代码块

```
1  always @(posedge clk)
2  begin
3      if (addr_err_load)
4          cause_exc_code_r <= 5'd4;      // Address Error Load
5      else if (addr_err_store)
6          cause_exc_code_r <= 5'd5;      // Address Error Store
7      else if ...
8  end
```

- 同步写入 `EPC`、置位 `STATUS.EXL`，`exc\_pc=0x0`，并发出 `cancel`。

代码块

```
1  begin
2      if (syscall | break_ex | reserved_ex | overflow_ex | addr_err_load |
    addr_err_store)
3      begin
4          epc_r <= pc;
5      end
6      else if (epc_wen)
7      begin
8          epc_r <= mem_result;
9      end
10 end
11 ...
```

```
12 assign cancel = (syscall | break_ex | reserved_ex | overflow_ex |  
    addr_err_load | addr_err_store | eret) & WB_over;
```

- 返回：异常处理程序可根据 CAUSE 区分 load/store 类型，处理后同样通过 `ERET` 返回。

3. BREAK 指令

触发条件：遇到 `BREAK`（R 型 `funct=001101`）。

- 产生位置与信号：
 - 在 `decode.v` 识别 `inst\_BREAK`，生成 `break\_ex` 并随总线下传；
 - 这里就是常规的添加break指令，然后一直传
 - EXE/MEM 透传至 WB。
- 异常确认：
 - `CAUSE.ExcCode <= 9`（Break），更新 `EPC/STATUS.EXL`，`exc\_pc=0x0`，发出 `cancel`。
 - 和上面的差不多，这里不再赘述
- 返回：异常处理程序可根据需要选择返回或停机，若返回则同样 EPC+4 后 `ERET`。

4. 保留指令（Reserved Instruction）

触发条件：解码阶段未匹配到任一已支持指令（剩余即视为保留/非法指令）。

- 产生位置与信号：
 - 在 `decode.v` 汇总所有已支持的 `inst\_xxx`，其余情况置 `reserved\_ex=1`，并随总线传递至 WB。

代码块

```
1 assign reserved_ex = ID_valid & ~  
    (inst_ADDU|inst_ADD|inst_SUBU|inst_SUB|inst_SLT|inst_SLTU|inst_JALR|inst_JR|  
    inst_AND|inst_NOR|inst_OR|inst_XOR|inst_SLL|inst_SLLV|inst_SRA|inst_SRAV|ins  
    t_SRL|inst_SRLV|inst_MULT|inst_MFLO|inst_MFHI|inst_MTLO|inst_MTHI|  
  
    inst_ADDIU|inst_ADDI|inst_SLTI|inst_SLTIU|inst_BEQ|inst_BGEZ|inst_BGTZ|inst_  
    BLEZ|inst_BLTZ|inst_BNE|  
    inst_LW|inst_SW|inst_LUI|inst_J|inst_LB|inst_LBU|inst_SB|  
  
    inst_ANDI|inst_ORI|inst_XORI|inst_JAL|inst_MFC0|inst_MTC0|inst_SYSCALL|inst_  
    ERET|inst_BREAK);
```

- 异常确认 (WB) :
 - `CAUSE.ExcCode <= 10` (Reserved Instruction) , 更新 `EPC/STATUS.EXL` ,
`exc\_pc=0x0` , 并 `cancel` 清空流水。
- 返回: 异常处理程序可记录并上报该非法指令来源, 再视策略决定是否返回。

5. 统一的异常收敛与跳转机制 (关键路径)

- 异常优先级 (在WB的时候) : `Address Error(load) > Address Error(store) > Syscall > Break > Reserved > Overflow` (已在 `wb.v` 的 CAUSE 更新逻辑中体现) 。
- EPC/STATUS 更新: 任一异常发生时 `EPC<=当前异常指令PC` , `STATUS.EXL<=1` ; 执行 `ERET` 或复位时 EXL 清零。
- 跳转与清空: `wb.v` 产生 `exc\_valid` 和 `exc\_pc` , `fetch.v` 在 `exc\_valid` 时优先取 `exc\_pc` ; 同时 `cancel` 清空各级 `\*\_valid` , 保证异常起跳的一致性。
- 返回路径: 异常处理程序 (放置于 `0x0`) 负责 `EPC+4` 修正、`ERET` 返回; 返回目标由 `exc\_pc=EPC` 决定。

四、波形图验证

为了验证上面的例外指令支持, 设计如下的COE指令文件:

起始pc值仍然为34, 由于这里都是为了展示触发例外指令而写的coe文件, 在支持之前肯定是译码部分都无法译码、整数溢出都是没有报错、地址错误的时候直接停止的。(很抱歉这里改完之后才想起来要展示修改之前的波形图, 改完之后没有保存之前的源码, 抱歉抱歉)

代码块

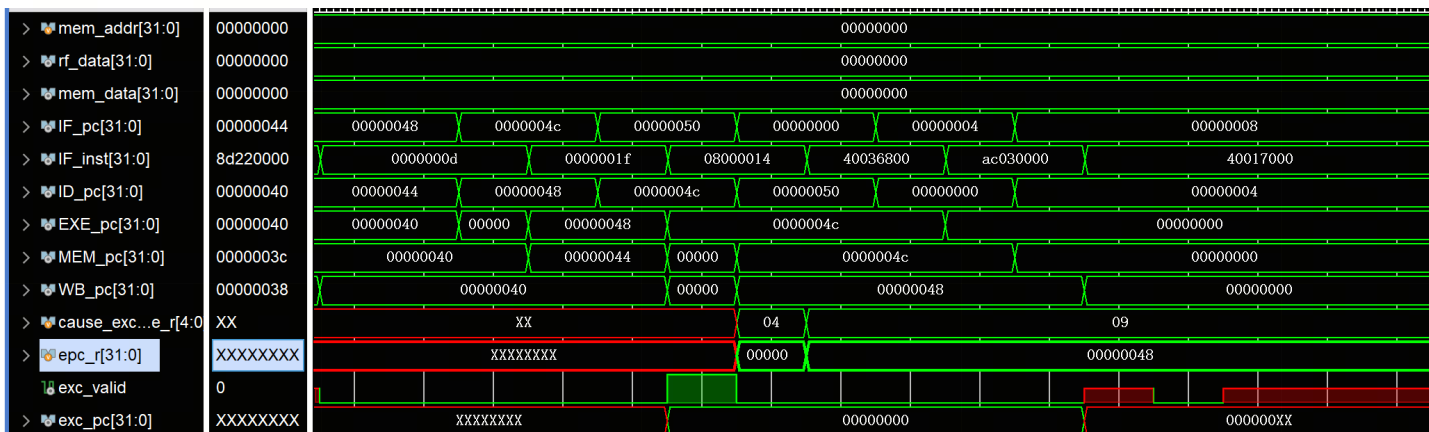
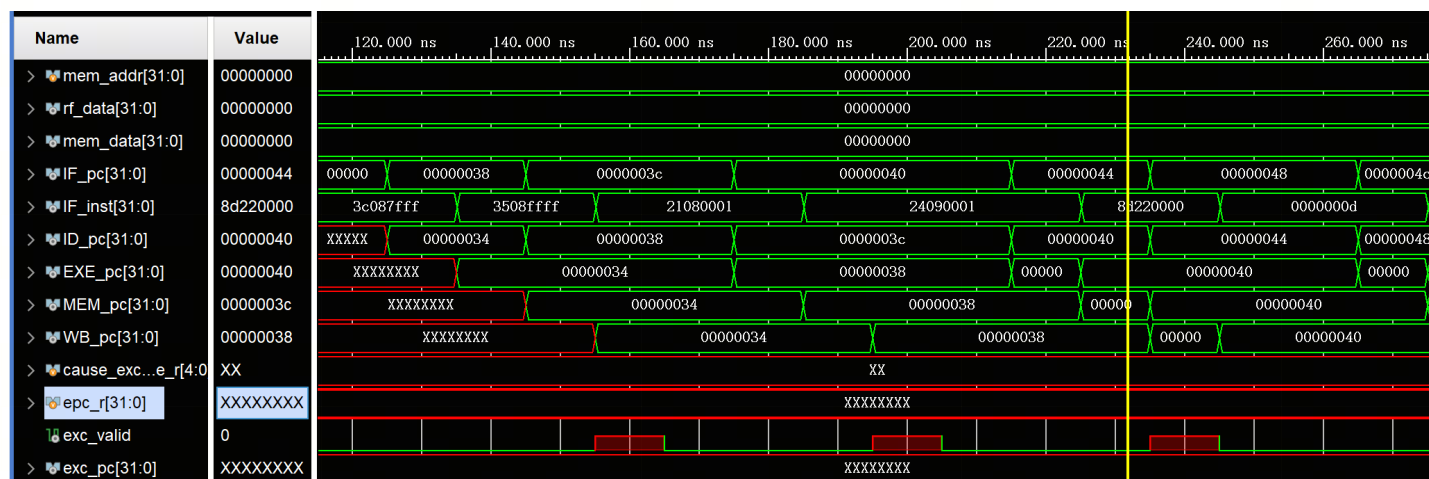
```
1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  40036800 // [0x00] mfc0 $3,$13 ; CAUSE -> r3
4  AC030000 // [0x04] sw $3,0($0) ; 存CAUSE到数据RAM[0]
5  40017000 // [0x08] mfc0 $1,$14 ; EPC -> r1
6  24210004 // [0x0C] addiu $1,$1,4 ; EPC += 4
7  40817000 // [0x10] mtc0 $1,$14 ; 写回 EPC
8  42000018 // [0x14] eret ; 返回
9  00000000 // [0x18] nop
10 00000000 // [0x1C] nop
11 00000000 // [0x20] nop
12 00000000 // [0x24] nop
13 00000000 // [0x28] nop
14 00000000 // [0x2C] nop
15 00000000 // [0x30] nop
```

```

16  3C087FFF // [0x34] LUI    $8,0x7FFF          ; t0 = 0x7FFF0000
17  3508FFFF // [0x38] ORI    $8,$8,0xFFFF          ; t0 = 0xFFFFFFFF
18  21080001 // [0x3C] ADDI   $8,$8,1              ; 触发整数溢出
19  24090001 // [0x40] ADDIU  $9,$0,1              ; t1 = 1 (奇地址)
20  8D220000 // [0x44] LW     $2,0($9)             ; 触发地址错 (未对齐)
21  0000000D // [0x48] BREAK                          ; 触发 break 异常
22  0000001F // [0x4C] 保留/非法指令                ; 触发 reserved 异常
23  08000014 // [0x50] j      0x00000014           ; 自循环, 避免跑飞
24  ;

```

可以看到波形图如下图所示

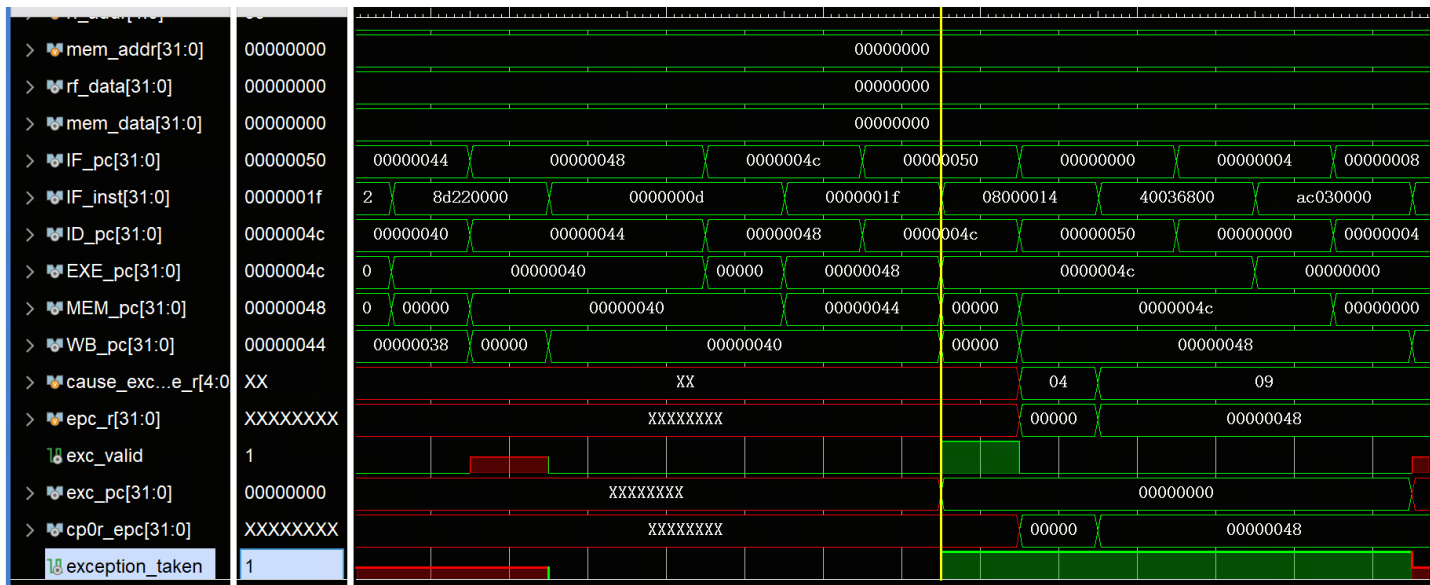


几点问题及分析：

1. 异常显示不完整

在3c处应该要有溢出，此时我们来观察第一个波形图，执行3c的时候就应该要显示异常，但是我debug了很久，每次这里exc_valid总是显示有问题，然后我们查找他的构成，发现总是有一些异常信号为X而导致这里无法返回异常信号。

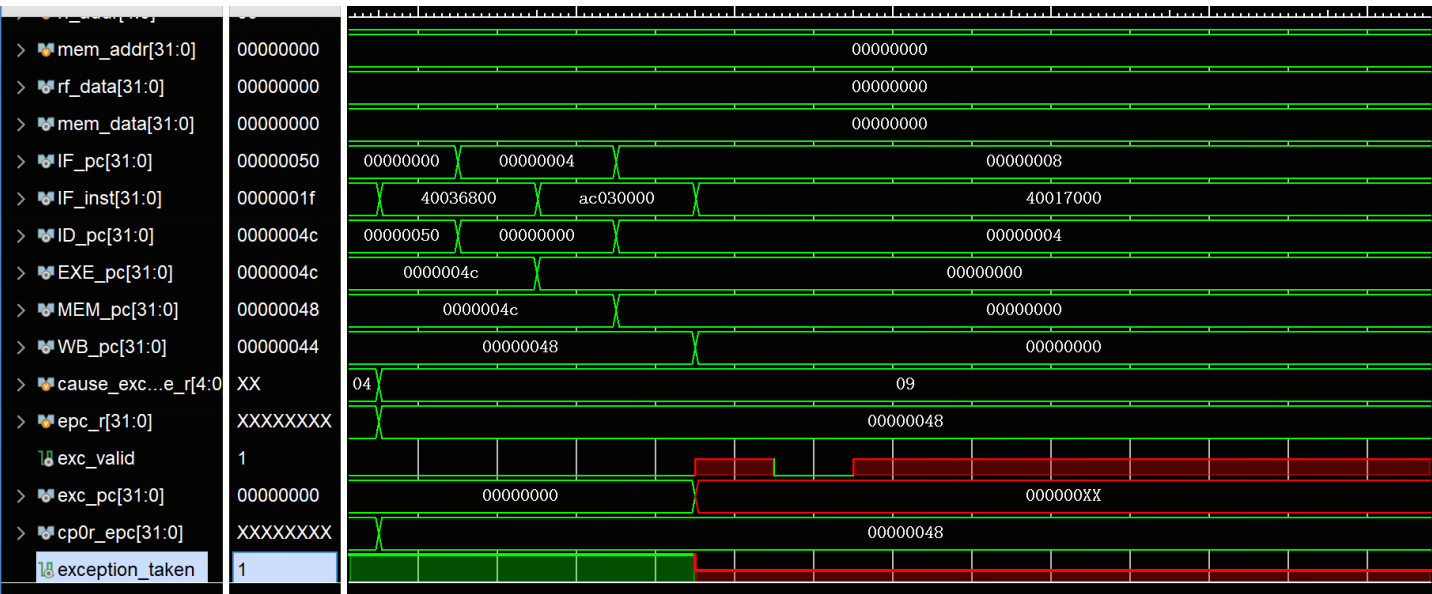
2. 异常跳转



wb执行到44的时候，触发了地址错误，此时pc跳转至异常入口00，进行环境的还原，以及异常的记录。

3. 无法eret回来

在进入异常跳转入口之后，本应该执行到30时跳转至原位置，但是发现在这里08一直卡死了可能是之前mfc0指令的实现有问题。



五、实验总结

感觉越来越跟不上这份代码了，之前自己实现的也有很多问题，导致后面越来越多，做作业难度很难，加上时间预留不是很多，本次实验完成比较失败，但是例外部分我感觉实现还比较完整。下一次实验换代码可能会好一点。